

PROJETO DE NAVEGAÇÃO E INTERAÇÃO

Prof. André Mussato

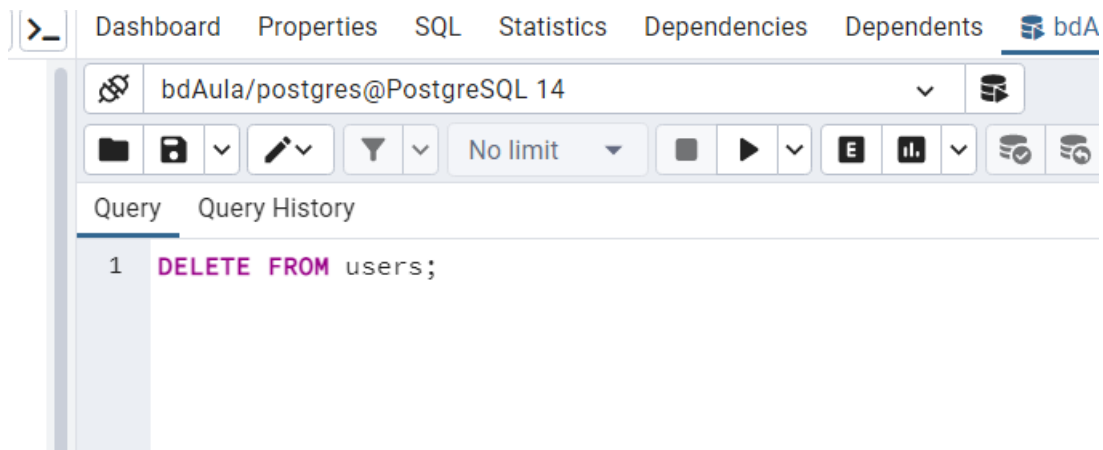
Apostila 2 – Criptografia da senha, Realizar Login Detalhes do Usuário Logado e Middleware de Autenticação

Projeto Pizzaria (parte 2):

Passo 1 — Remover usuários cadastrados

Dentro do **pgAdmin**, acesse o banco de dados criado e execute o seguinte comando SQL:

```
DELETE FROM users;
```



Para que serve esse comando?

Esse comando remove todos os registros da tabela **users**, ou seja, apaga todos os usuários cadastrados no banco de dados.

Ele é útil para:

- limpar a base de dados durante testes
- evitar conflitos (ex: emails duplicados)
- reiniciar o estado da aplicação

Atenção: esse comando não pode ser desfeito e remove permanentemente todos os dados da tabela.

Passo 2 — Instalar o bcryptjs

Ainda dentro da pasta **backend**, execute o comando:

`npm install bcryptjs`

```
PS C:\Users\andre\Desktop\Curso Full Stack\pizzaria\backend> npm install bcryptjs

added 1 package, and audited 206 packages in 4s

37 packages are looking for funding
  run `npm fund` for details

11 vulnerabilities (5 moderate, 6 high)

To address issues that do not require attention, run:
  npm audit fix

To address all issues (including breaking changes), run:
  npm audit fix --force

Run `npm audit` for details.
PS C:\Users\andre\Desktop\Curso Full Stack\pizzaria\backend> 
```

Para que serve esse comando?

Esse comando instala a biblioteca **bcryptjs**, utilizada para criptografar senhas antes de armazená-las no banco de dados.

Com ela, você pode:

- proteger as senhas dos usuários (não salvar em texto puro)
- aumentar a segurança da aplicação
- evitar que dados sensíveis sejam expostos em caso de vazamento

O **bcrypt** transforma a senha em um **hash criptografado**, tornando impossível recuperar a senha original diretamente.

Passo 3 — Criptografar a senha do usuário

Agora, dentro do arquivo `CreateUserService.ts`, altere o código para incluir a criptografia da senha antes de salvá-la no banco de dados:

TS CreateUserService.ts X

src > services > user > TS CreateUserService.ts > CreateUserService > execute

```
1
2 import prismaClient from '../prisma/index';
3
4 // Importa a função hash do bcryptjs para criptografar a senha do usuário
5 import { hash } from 'bcryptjs';
6
7 // Interface que define os dados necessários para criar um usuário
8 interface CreateUserProps {
9   name: string;
10  email: string;
11  password: string;
12 }
13
14 // Define uma classe responsável pela lógica de criação de usuário
15 class CreateUserService {
16
17   // Método assíncrono (permite usar await futuramente, como acesso a banco)
18   async execute({ name, email, password }: CreateUserProps) {
19
20     // Busca no banco se já existe um usuário com o mesmo email
21     const userAlreadyExist = await prismaClient.user.findFirst({
22       where: {
23         email: email
24       }
25     })
26
27     // Verifica se o usuário já existe
28     if (userAlreadyExist) {
29
30       // Lança um erro para impedir a criação de usuário duplicado
31       throw new Error("Usuário já existente!");
32
33       // ⚠ Este console.log nunca será executado (código morto),
34       // pois o throw encerra a execução imediatamente
35       console.log("Erro! Usuário já existe!");
36     }
37   }
38 }
```

```

33
34     // ⚠ Esse console.log nunca será executado (código morto),
35     // pois o throw encerra a execução imediatamente
36     console.log("Erro! Usuário já existe!");
37 }
38
39 // Criptografa a senha antes de salvar no banco (boa prática de segurança)
40 const passwordHash = await hash(password, 8);
41
42 // Cria um novo usuário no banco de dados
43 const user = await prismaClient.user.create({
44   data: {
45     name: name,
46     email: email,
47     // Salva a senha já criptografada, nunca a senha original
48     password: passwordHash,
49   }
50 })
51
52 // Retorna o nome do usuário criado
53 return user;
54
55 }
56 }
57
58 // Exporta a classe para que possa ser usada em outros arquivos (ex: controllers)
59 export { CreateUserService };

```

Para que serve esse comando?

Essa alteração adiciona uma camada essencial de segurança na aplicação.

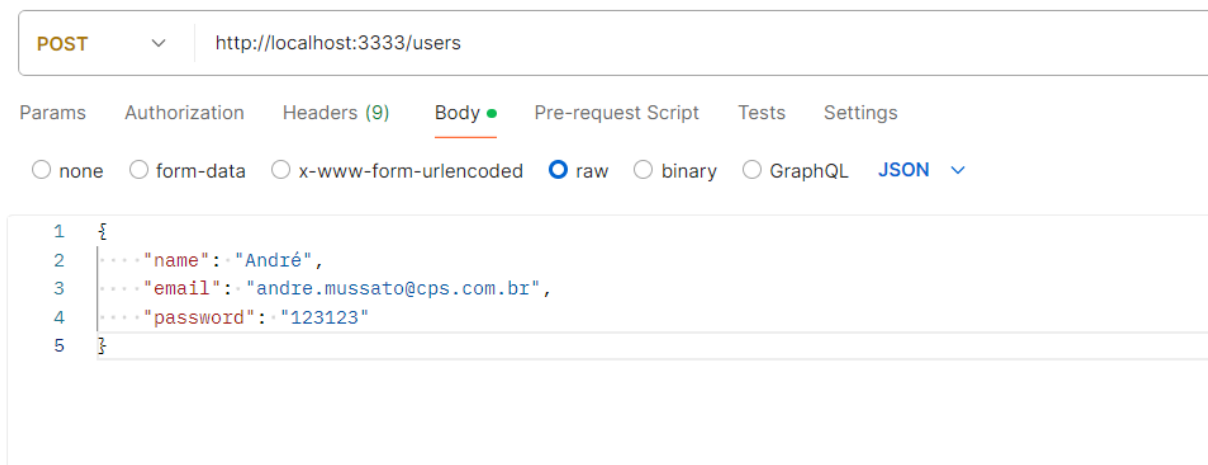
Agora:

- a senha não é mais salva em texto puro
- é armazenada como um **hash criptografado**
- mesmo que o banco seja acessado, a senha original não pode ser facilmente descoberta

Isso é uma prática fundamental em qualquer sistema que lida com autenticação de usuários.

Passo 4 — Testar cadastro com senha criptografada

Agora, utilize o Postman para cadastrar um novo usuário e verificar se a senha está sendo criptografada corretamente.



Passo 5 — Ajustar o controller de usuário

Agora, no arquivo `CreateUserController.ts`, faça duas alterações simples:

Comente o `console.log` (usado apenas para debug)

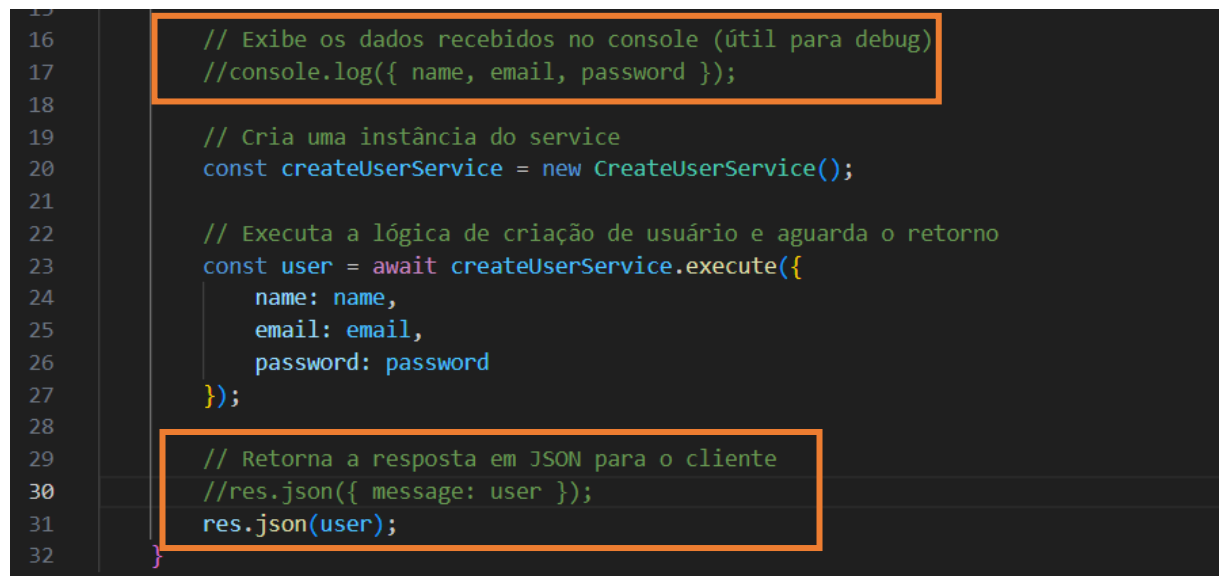
Ajuste o retorno da resposta

O que alterar?

`// console.log({ name, email, password });` // Comentado pois não é mais necessário

E ajuste o retorno para algo mais adequado:

`res.json({ user });`



Passo 6 — Ajustar o Service para retornar dados específicos

Agora, no arquivo `CreateUserService.ts`, adicione a propriedade `select` na criação do usuário.

```
50      //O select define exatamente quais campos do banco serão retornados após a operação.
51      //Ou seja, exibidos no postman
52      select: {
53        id: true,
54        name: true,
55        email: true,
56        role: true,
57        createdAt: true,
58      }
59    })
60  })
61
62  // Retorna o nome do usuário criado
63  return user;
64
```

Para que serve essa alteração?

Essa modificação permite controlar exatamente quais dados serão retornados após a criação do usuário.

Com isso:

- evita expor informações sensíveis (como a senha)
- retorna apenas os dados necessários para o cliente
- melhora a organização e segurança da API
- realize um novo teste no Postman, cadastrando um usuário. Agora você perceberá que: a senha não será mais exibida na resposta

Passo 7 – Adicionar validação de login no schema

Agora vamos criar uma nova validação para autenticação de usuário (login).

No arquivo `userSchema.ts`, adicione o seguinte código:

```
31  // Cria um schema de validação para login de usuário
32  export const authUserSchema = z.object({
33    body: z.object({
34      // Define que deve ser uma string no formato de email válido
35      email: z.email({ message: "Precisa ser um email válido" }),
36      password: z.
37        string({ message: "A senha é obrigatória" })
38        // Define que deve ter no mínimo 1 caracter
39        .min(1, { message: "A senha é obrigatória" })
40    })
41  })
```

Passo 8 — Gerar chave secreta (JWT_SECRET)

Agora é necessário criar uma chave secreta que será usada para gerar os tokens de autenticação.

Acesse o site:

<https://www.md5hashgenerator.com/>

Digite uma palavra (ex: fatecjales) e gere um hash MD5.

MD5 Hash Generator

Use this generator to create an MD5 hash of a string:

fatecjales

Generate →

Your String	fatecjales
MD5 Hash	33078f70cbc104fd6f13685ce303ad77 <button>Copy</button>
SHA1 Hash	7b248715cfa038ab138dc7d0e3948e72f1f57047 <button>Copy</button>

Passo 9 - Configurar no .env

Após gerar o hash, adicione no seu arquivo .env:

JWT_SECRET="hash_gerado_aqui"

```
16 # - nome do banco (bdAula)
17 # - schema utilizado (public)
18 DATABASE_URL="postgres://postgres:postdba@localhost:5433/bdAula?schema=public"
19
20 # Essa chave (JWT_SECRET) é usada para assinar e validar os tokens JWT.
21 JWT_SECRET="33078f70cbc104fd6f13685ce303ad77"
```

Para que serve isso?

Essa chave (JWT_SECRET) é usada para **assinar e validar os tokens JWT**.

Quando o usuário faz login:

- o sistema gera um **token**
- esse token contém dados do usuário (como id, email, etc.)
- o token é assinado com essa chave secreta

Quando o usuário faz novas requisições:

- o sistema verifica se o token é válido usando essa mesma chave

Sobre a “senha” (JWT_SECRET)

- isso **não é uma senha de usuário**
- é uma **chave secreta do sistema**

Ela serve para:

- garantir que o token não foi alterado
- impedir que terceiros criem tokens falsos

Importante:

- nunca compartilhe essa chave
- não suba ela no GitHub
- use sempre no .env

Passo 10 – Instalar o jsonwebtoken

Com a pasta **backend** aberta no terminal, execute os seguintes comandos:

```
npm install jsonwebtoken
```

```
npm i --save-dev @types/jsonwebtoken
```

```
PS C:\Users\andre\Desktop\Curso Full Stack\teste\pizzaria\backend> npm install jsonwebtoken

11 vulnerabilities (5 moderate, 6 high)

To address issues that do not require attention, run:
  npm audit fix

To address all issues (including breaking changes), run:
  npm audit fix --force

Run `npm audit` for details.
PS C:\Users\andre\Desktop\Curso Full Stack\teste\pizzaria\backend> npm i --save-dev @types/jsonwebtoken

added 2 packages, and audited 222 packages in 3s

38 packages are looking for funding
  run `npm fund` for details

11 vulnerabilities (5 moderate, 6 high)

To address issues that do not require attention, run:
  npm audit fix
```

Para que servem esses comandos?

- **jsonwebtoken**: biblioteca responsável por criar e validar tokens JWT
- **@types/jsonwebtoken**: adiciona tipagem para usar a biblioteca com TypeScript

Por que isso é importante?

Essa biblioteca permite:

- gerar tokens quando o usuário faz login
- validar tokens em rotas protegidas
- controlar autenticação na aplicação

Ou seja, ela é essencial para implementar o sistema de login seguro usando JWT.

Passo 11 — Criar serviço de autenticação

Agora, dentro da pasta `services/user`, crie um novo arquivo chamado `AuthUserService.ts`.

Em seguida, implemente o código de autenticação do usuário.

```
src > services > user > TS AuthUserService.ts > ...
 1  // Importa a função compare do bcryptjs
 2  // Serve para comparar a senha digitada com a senha criptografada
 3  import { compare } from "bcryptjs";
 4
 5  // Importa a função sign do jsonwebtoken
 6  // Serve para gerar um token JWT (autenticação)
 7  import { sign } from "jsonwebtoken";
 8
 9  // Importa o cliente do Prisma (conexão com o banco de dados)
10  import prismaClient from "../prisma/index";
11
12  // Interface que define o tipo dos dados recebidos
13  interface AuthUserServiceProps {
14    email: string;
15    password: string;
16  }
17
18  // Classe responsável pela autenticação do usuário
19  class AuthUserService {
20
21    // Método principal que executa a autenticação
22    async execute({ email, password }: AuthUserServiceProps) {
23
24      // Busca um usuário no banco pelo email
25      const user = await prismaClient.user.findFirst({
26        where: {
27          email: email,
28        },
29      });
30    }
```

```

30
31 // Se não encontrar o usuário, retorna erro
32 if (!user) {
33   throw new Error("Email/Senha é obrigatório");
34 }
35
36 // Verifica se a senha digitada é igual à senha criptografada do banco
37 const passwordMatch = await compare(password, user.password);
38
39 // Se a senha estiver incorreta, retorna erro
40 if (!passwordMatch) {
41   throw new Error("Email/Senha é obrigatório");
42 }
43
44 // GERAR TOKEN JWT
45 // O token é usado para autenticar o usuário nas próximas requisições
46 const token = sign(
47   {
48     // Dados que vão dentro do token (payload)
49     name: user.name,
50     email: user.email,
51   },
52   process.env.JWT_SECRET as string, // chave secreta (fica no .env)
53   {
54     subject: user.id, // identifica o usuário no token
55     expiresIn: "30d", // tempo de expiração (30 dias)
56   }
57 );
58

```

```

59 // Retorna os dados do usuário + token
60 return {
61   id: user.id,
62   name: user.name,
63   email: user.email,
64   role: user.role,
65   token: token,
66 };
67 }
68 }
69
70 // Exporta a classe para ser usada em outros arquivos
71 export { AuthUserService };

```

Passo 12 — Criar controller de login

Agora, dentro da pasta controllers/user, crie o arquivo AuthUserController.ts e implemente o seguinte código:

```

src > controllers > user > TS AuthUserController.ts > ...
1  // Importa os tipos Request e Response do Express
2  // Request representa a requisição (dados que chegam)
3  // Response representa a resposta (dados que serão enviados)
4  import { Request, Response } from "express";
5
6  // Importa o serviço responsável pela autenticação do usuário
7  import { AuthUserService } from "../../services/user/AuthUserService";
8
9  // Cria a classe do controller de autenticação
10 class AuthUserController {
11
12     // Método que será chamado quando a rota de login for acessada
13     async handle(req: Request, res: Response) {
14
15         // Extrai email e senha do corpo da requisição (body)
16         const { email, password } = req.body;
17
18         // Instancia o serviço de autenticação
19         const authService = new AuthUserService();
20
21         // Executa o serviço passando email e senha
22         // Esse método vai validar usuário e gerar o token
23         const session = await authService.execute({ email, password });
24
25         // Retorna os dados (incluindo o token) em formato JSON
26         res.json(session);
27     }
28 }
29
30 /* DICA: Controller = recebe a requisição
31 Service = faz a lógica (validação, banco, etc)
32 */
33
34 // Exporta o controller para ser usado nas rotas
35 export { AuthUserController };

```

Para que serve esse código?

Esse controller é responsável por lidar com o processo de login na aplicação.
Ele:

- recebe email e senha da requisição
- chama o service de autenticação
- retorna os dados do usuário + token JWT

Resumo

- **Controller** → recebe a requisição
- **Service** → executa a lógica (validação + token)

Ou seja, esse passo conecta o login do usuário com o sistema de autenticação da API.

Passo 13 – Implementar rota de login

Agora, no arquivo routes.ts, adicione a rota responsável pela autenticação do usuário:

```
32
33 // Define uma rota do tipo POST no caminho "/session"
34 // Essa rota será usada para autenticação (login do usuário)
35 router.post(
36   "/session", // endpoint da rota (ex: http://localhost:3000/session)
37   // Middleware que valida os dados enviados no body da requisição
38   // Ele usa um schema (authUserSchema) para garantir que email e senha estão corretos
39   validateSchema(authUserSchema),
40   // Controller responsável por processar a requisição
41   // Aqui ele executa o método handle, que faz a autenticação do usuário
42   new AuthUserController().handle
43 );
44
45 // Exporta o router para ser utilizado em outros arquivos (ex: no app principal)
46 export { router };
```

Não esqueça das importações:

```
1 // Importa os tipos Request e Response (para tipagem das requisições e respostas)
2 // e o Router (para criar rotas separadas no Express)
3 import { Request, Response, Router } from "express";
4
5 // Importa o controller responsável por lidar com a criação de usuários
6 import { CreateUserController } from "../controllers/user/CreateUserController";
7
8 // Importa o controller responsável por lidar com o login de usuários
9 import { AuthUserController } from "../controllers/user/AuthUserController";
10
11 // Importa o middleware de validação com Zod
12 import { validateSchema } from "../middlewares/validateSchema";
13
14 // Importa o schema que define as regras de validação para criação de usuário
15 import { createUserSchema, authUserSchema } from "../schemas/userSchema";
16
```

Para que serve esse código?

Essa rota permite que o usuário faça login na aplicação.

Ela funciona assim:

1. Recebe email e senha via requisição
2. Valida os dados com o schema (Zod)
3. Chama o controller de autenticação
4. Retorna um token JWT se estiver tudo correto

Resumo

Essa etapa conecta tudo:

- **Rota** → define o endpoint /session
- **Middleware** → valida os dados
- **Controller** → chama o service

- **Service** → autentica e gera o token
- Ou seja, aqui o fluxo completo de login passa a funcionar.

Passo 14 — Adicionar middleware global de erros

Agora, no arquivo `server.ts`, adicione o middleware global de tratamento de erros:

Altere a importação:

```
src > TS server.ts > ...
1  // Importa o middleware CORS (permite requisições de diferentes origens
2  import cors from "cors";
3
4  // Carrega automaticamente as variáveis de ambiente do arquivo .env
5  import "dotenv/config";
6
7  // Importa o Express (framework para criar o servidor)
8  import express, { NextFunction, Request, Response } from 'express'
9
10 // Importa as rotas definidas em outro arquivo
11 import { router } from "../routes";
```

E adicione:

```
28
29 // Inicia o servidor e exibe uma mensagem no console quando estiver rodando
30 app.listen(PORT, () => {
31   console.log("Servidor rodando na porta " + PORT);
32 });
33
34 // Middleware global de tratamento de erros do Express
35 // Ele captura qualquer erro lançado na aplicação (ex: throw new Error)
36 app.use((error: Error, _: Request, res: Response, next: NextFunction) => {
37
38   // Verifica se o erro é uma instância da classe Error
39   if(error instanceof Error) {
40
41     // Retorna status 400 (erro do cliente) com a mensagem do erro
42     return res.status(400).json({
43       error: error.message,
44     });
45   }
46
47   // Caso não seja um erro conhecido, retorna erro genérico do servidor
48   return res.status(500).json({
49     error: "Internal server error!"
50   });
51 });
```

Para que serve esse código?

Esse middleware é responsável por tratar erros de forma centralizada na aplicação.

Ele:

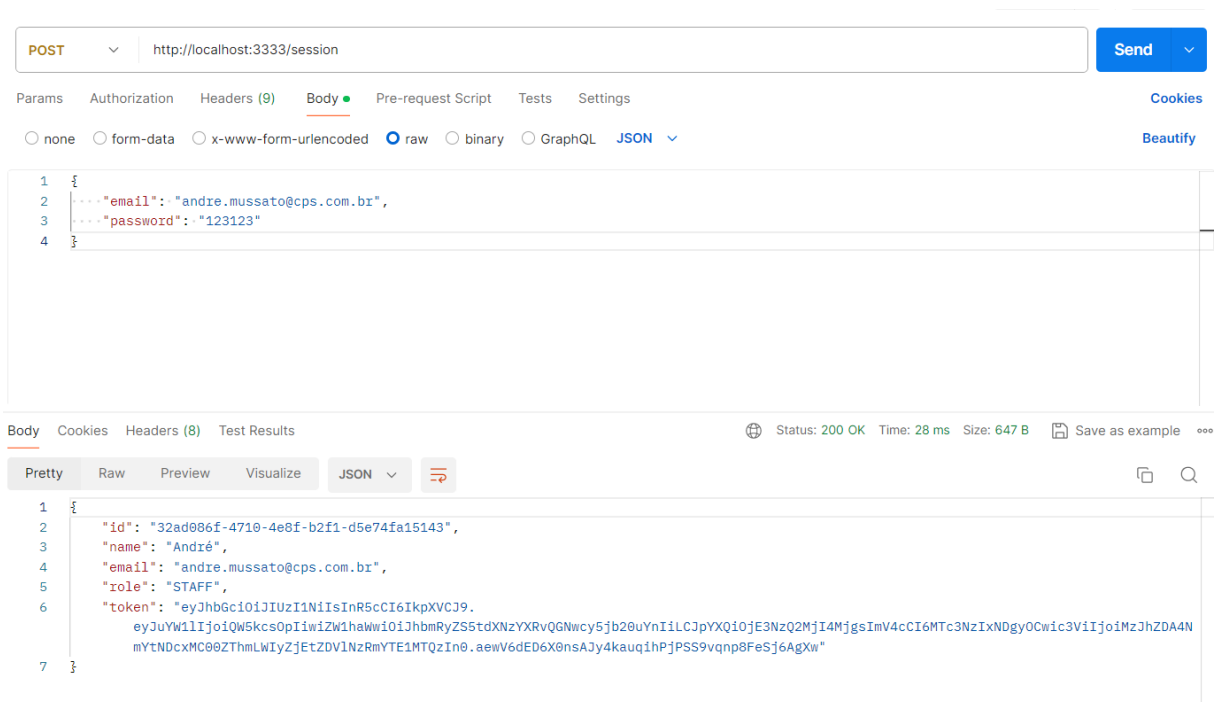
- captura erros lançados com `throw new Error()`
- evita que a aplicação quebre
- padroniza as respostas de erro

Como funciona?

- Se for um erro conhecido → retorna 400 + mensagem
- Se for um erro desconhecido → retorna 500 (erro interno)

Passo 15 – Testar login no Postman

Agora abra o Postman e realize uma requisição para testar o login do usuário:



Passo 16 – Criar tipagem personalizada do Express

Agora, dentro da pasta **src**, crie a seguinte estrutura:

- **@types/**
 - **express/**
 - **index.d.ts**

Dentro do arquivo **index.d.ts**, adicione o seguinte código:

```
src > @types > express > TS index.d.ts > {} Express
1 declare namespace Express {
2     // Aqui estamos "reabrindo" (extendendo) o namespace do Express
3     // O TypeScript permite adicionar novas propriedades a tipos já existentes (chamado de Declaration Merging)
4
5     export interface Request {
6         user_id: string;
7         // Estamos adicionando uma nova propriedade ao objeto Request do Express
8         // Ou seja, agora toda requisição (req) pode ter um campo "user_id"
9
10        // Isso é muito útil quando usamos autenticação
11        // Exemplo: um middleware (como o isAuthenticated) adiciona o user_id no req
12
13        // Sem isso, o TypeScript daria erro:
14        // "Property 'user_id' does not exist on type Request"
15    }
16 }
```

Para que serve esse código?

Esse arquivo estende a tipagem padrão do Express, permitindo adicionar propriedades personalizadas ao objeto Request.

Agora:

- o req.user_id passa a ser reconhecido pelo TypeScript
- evita erros de tipagem durante o desenvolvimento
- permite compartilhar dados entre middlewares e controllers

Passo 17 – Criar middleware de autenticação

Agora, dentro da pasta middlewares, crie o arquivo isAuthenticated.ts e adicione o seguinte código:

```
src > middlewares > TS isAuthenticated.ts > ...
1  import { Request, Response, NextFunction } from 'express'
2  // Importa os tipos do Express:
3  // Request → dados da requisição
4  // Response → resposta que será enviada
5  // NextFunction → função usada para passar o controle para o próximo middleware
6
7  import { verify } from 'jsonwebtoken'
8  // Importa a função verify da biblioteca jsonwebtoken
9  // Ela é responsável por validar e decodificar o token JWT
10
11 interface Payload {
12   sub: string;
13 }
14 // Define a estrutura esperada do payload do token JWT
15 // "sub" (subject) normalmente guarda o ID do usuário
16
17 export function isAuthenticated(req: Request, res: Response, next: NextFunction) {
18   // Cria um middleware chamado isAuthenticated
19   // Ele será usado para proteger rotas (ex: só usuários logados podem acessar)
20
21   const authToken = req.headers.authorization
22   // Pega o header "Authorization" da requisição
23   // Normalmente vem assim: "Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
24
25   if (!authToken) {
26     // Verifica se o token foi enviado
27
28     return res.status(401).json({
29       error: "Token não fornecido"
30     })
31     // Retorna erro 401 (não autorizado) se não houver token
32   }
33 }
```

```

33
34     const [, token] = authToken.split(" ");
35     // Divide o header em duas partes: "Bearer" e o token
36     // Ignora a primeira parte e pega apenas o token
37     // Exemplo: ["Bearer", "TOKEN_AQUI"]
38
39     try {
40         // Tenta validar o token
41
42         const { sub } = verify(
43             token!,
44             process.env.JWT_SECRET as string
45         ) as Payload
46         // Verifica se o token é válido usando a chave secreta (JWT_SECRET)
47         // Se for válido, extrai o payload
48         // Aqui estamos pegando o "sub", que é o ID do usuário
49
50         req.user_id = sub;
51         // Adiciona o ID do usuário dentro do objeto req
52         // Isso permite acessar o user_id em qualquer rota protegida
53
54         return next();
55         // Chama o próximo middleware ou controller
56         // Isso significa: "autenticação OK, pode continuar"
57
58     } catch (erro) {
59         // Se o token for inválido, expirado ou mal formado
60
61         return res.status(401).json({
62             error: "Token inválido"
63         })
64         // Retorna erro 401 (não autorizado)
65     }
66 }

```

Para que serve esse código?

Esse middleware protege rotas da aplicação, garantindo que apenas usuários autenticados possam acessá-las.

Ele:

- verifica se o token JWT foi enviado
- valida o token usando a chave secreta
- extrai o ID do usuário do token
- libera o acesso apenas se o token for válido

Passo 18 – Criar serviço de detalhamento do usuário

Agora, dentro da pasta `services/user`, crie o arquivo `DetailUserService.ts` e adicione o seguinte código:

```
1
2 import prismaClient from "../../prisma"
3 // Importa a instância do Prisma Client
4 // Essa instância é responsável por fazer a comunicação com o banco de dados
5
6 class DetailUserService {
7 // Define uma classe chamada DetailUserService
8 // Ela segue o padrão de "Service", ou seja, responsável pela regra de negócio
9
10   async execute(user_id: string) {
11     // Método assíncrono chamado "execute"
12     // Recebe como parâmetro o ID do usuário (user_id)
13     // O async permite usar await dentro da função
14
15     try {
16       // Início do bloco de tentativa (try)
17       // Aqui tentamos executar a busca no banco de dados
18
19       const user = await prismaClient.user.findFirst({
20         // Usa o Prisma para buscar um usuário na tabela "user"
21         // findFirst retorna o primeiro registro que bater com a condição
22
23         where: {
24           id: user_id
25         },
26         // Define o filtro da busca
27         // Aqui estamos procurando um usuário com o ID recebido
28
29         select: {
30           id: true,
31           name: true,
32           email: true,
33           role: true,
34           createdAt: true,
35         }
36       })
37     }
38   }
39 }
```

```

35     }
36     // Define quais campos queremos retornar do banco
37     // Isso evita trazer dados desnecessários (boa prática)
38 })
39
40 if (!user) {
41     // Verifica se nenhum usuário foi encontrado
42
43     throw new Error("Usuário não encontrado")
44     // Lança um erro caso não encontre o usuário
45     // Esse erro pode ser tratado no controller ou middleware global
46 }
47
48 return user;
49 // Retorna o usuário encontrado
50
51 } catch (erro) {
52     // Caso aconteça qualquer erro dentro do try
53
54     console.log(erro);
55     // Mostra o erro no console (útil para debug)
56
57     throw new Error("Usuário não encontrado")
58     // Lança um erro genérico
59     // OBS: aqui você perde o erro original (não é ideal em produção)
60 }
61
62 }
63 }
64
65 export { DetailUserService }
66 // Exporta a classe para poder ser usada em outros arquivos (ex: controller)
67

```

Para que serve esse código?

Esse service é responsável por buscar e retornar os dados de um usuário específico no banco de dados.

Ele:

- recebe o ID do usuário
- consulta o banco utilizando o Prisma
- retorna apenas os campos necessários (usando select)
- trata erros caso o usuário não exista

Passo 19 – Criar serviço de controle dos detalhes de usuário

Agora, dentro da pasta controllers/user, crie o arquivo DetailUserController.ts e adicione o seguinte código:

```
src > controllers > user > TS DetailUserController.ts > ...
1  import { Request, Response } from 'express';
2  // Importa os tipos Request e Response do Express
3  // Request representa a requisição recebida
4  // Response representa a resposta que será enviada ao cliente
5
6  import { DetailUserService } from '../services/user/DetailUserService'
7  // Importa o service responsável por buscar os detalhes do usuário no banco de dados
8
9  class DetailUserController {
10 // Cria a classe DetailUserController
11 // O Controller é responsável por receber a requisição e devolver uma resposta
12
13   async handle(req: Request, res: Response) {
14     // Cria o método assíncrono handle
15     // Esse método será chamado quando a rota de detalhes do usuário for acessada
16     // O async permite usar await dentro dele
17
18     const user_id = req.user_id;
19     // Pega o ID do usuário que está salvo dentro da requisição
20     // Normalmente esse req.user_id é adicionado por um middleware de autenticação
21     // Exemplo: o middleware valida o token JWT e coloca o ID do usuário em req.user_id
22
23     const detailUser = new DetailUserService();
24     // Cria uma instância da classe DetailUserService
25     // Esse service contém a regra de negócio para buscar os dados do usuário
26
27     const user = await detailUser.execute(user_id);
28     // Chama o método execute do service, passando o ID do usuário
29     // O await espera a busca no banco de dados terminar
30     // O resultado será armazenado na variável user
31
32     return res.json(user)
33     // Retorna os dados do usuário em formato JSON como resposta da API
34   }
35 }
36
37 export { DetailUserController };
```

Para que serve esse código?

Esse service é responsável por buscar e retornar os dados de um usuário específico no banco de dados.

Ele:

- recebe o ID do usuário
- consulta o banco utilizando o Prisma
- retorna apenas os campos necessários (usando select)
- trata erros caso o usuário não exista

Passo 20 – Adicionar rota de detalhes do usuário

Agora, no arquivo **routes.ts**, adicione a rota responsável por buscar os dados do usuário logado:

Não esqueça das importações:

```
17 import { isAuthenticated } from "../middlewares/isAuthenticated";
18 import { DetailUserController } from "../controllers/user/DetailUserController";
19
50
51 router.get("/me", // Endpoint para obter dados do usuário autenticado
52   isAuthenticated, // Middleware que verifica se o usuário está autenticado (token válido)
53   new DetailUserController().handle // Controller que busca e retorna os dados do usuário
54 );
55
56 // Exporta o router para ser utilizado em outros arquivos (ex: no app principal)
57 export { router };
```

Para que serve esse código?

Essa rota permite que o usuário autenticado consulte seus próprios dados.

Ela funciona assim:

1. Verifica se o usuário está autenticado (token JWT)
2. Extrai o `user_id` do token
3. Busca os dados do usuário no banco
4. Retorna as informações em JSON

Passo 20 — Testar rota de detalhes no Postman

Realizar o teste da rota protegida que retorna os dados do usuário autenticado.

Configuração no Postman

- **Método:** GET
- **URL:** `http://localhost:3333/me`

Autenticação (IMPORTANTE)

Essa rota é protegida, portanto é necessário enviar o token JWT.

No Postman:

1. Acessar a aba **Authorization**
2. Selecionar o tipo **Bearer Token**
3. Inserir o token obtido no login

Ou via header:

Authorization: Bearer SEU_TOKEN_AQUI (O token deve ser copiado após realizar a rota de login também pelo postman)

[illegible]

The screenshot displays the Pizzaria API client interface. At the top, the URL bar shows a GET request to `http://localhost:3333/me`. Below the URL bar, the 'Headers' tab is active, showing an 'Authorization' header with a Bearer token. The 'Body' tab is also visible, showing the JSON response of the user profile.

Headers:

Key	Value	Description
☒ Authorization	Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJuYW...	
Key	Value	Description

Body:

```

1 {
2   "id": "8a5804b0-c7d5-459f-a63d-c6e905e8160d",
3   "name": "André",
4   "email": "andree.mussato@cps.com",
5   "role": "STAFF",
6   "createAt": "2026-03-31T22:47:36.441Z"
7 }
```